PROJECT REPORT

AI PATHFINDING VISUALIZER

NAME – Tanish Paharia
REG. NO. – 25BCE10219
DEPT. – BTECH CSE CORE

INTRODUCTION :

This project is an interactive Pathfinding Visualizer developed using Python and Pygame. It demonstrates how Artificial Intelligence algorithms are used to find optimal paths between two points in a grid-based environment.

The system allows users to create obstacles and visualize how different search algorithms explore the grid and determine the best possible path. It provides a practical understanding of AI concepts like search strategies and heuristics.

PROBLEM STATEMENT :

In real-world scenarios such as navigation systems, finding the shortest and most efficient route is a major challenge.

Manual understanding of such processes is difficult, and most users are unaware of how systems like GPS compute routes. Therefore, a system is required to simulate and visualize intelligent pathfinding algorithms to improve understanding and efficiency.

FUNCTIONAL REQUIREMENTS :

1. Create a grid-based environment
2. Select start and end nodes
3. Add obstacles (barriers)
4. Execute pathfinding algorithms:
    - Breadth-First Search (BFS)
    - Depth-First Search (DFS)
    - A* Algorithm
5. Visualize algorithm execution step-by-step
6. Display final shortest path

NON-FUNCTIONAL REQUIREMENTS :

1. **Performance:** Smooth real-time visualization
2. **Usability:** Simple and intuitive controls
3. **Reliability:** Accurate pathfinding results
4. **Scalability:** Adjustable grid size
5. **Maintainability:** Modular and readable code

SYSTEM ARCHITECTURE

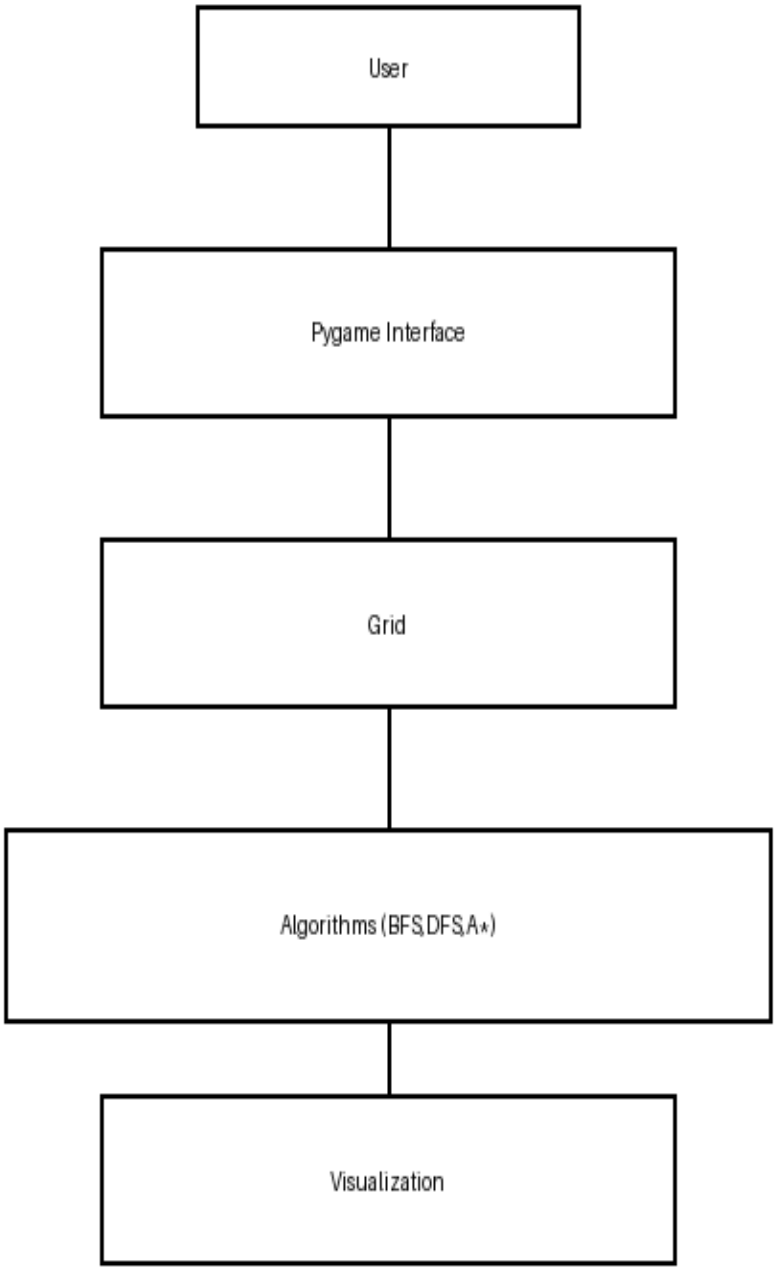
The system follows a grid-based architecture where each cell is treated as a node in a graph.

- User Interface: Pygame window
- Processing: AI search algorithms
- Data Representation: 2D grid (matrix)

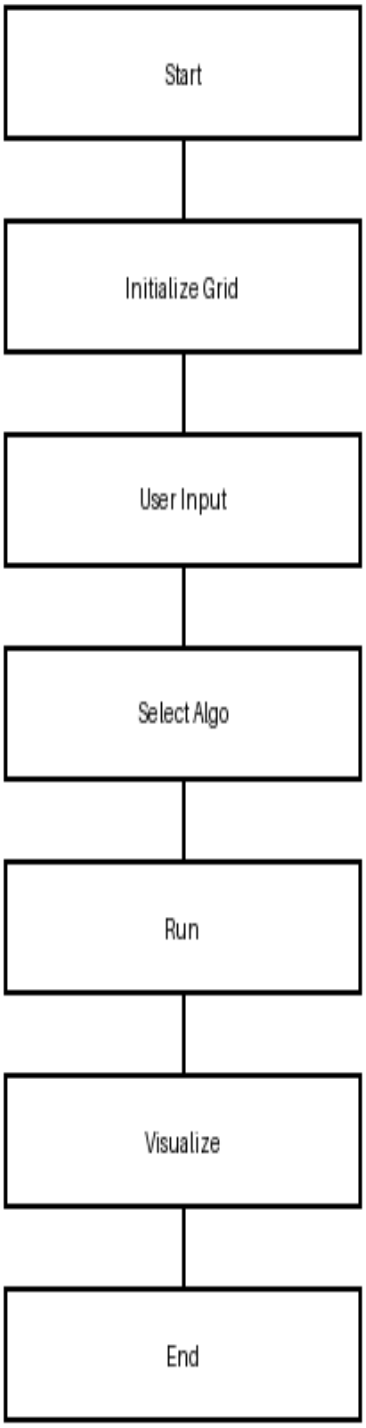
The architecture is similar to the CLI-based system in the reference project , but enhanced with graphical visualization.

DESIGN DIAGRAMS

USE CASE DIAGRAM (Conceptual)



WORKFLOW DIAGRAM :



DESIGN DECISIONS & RATIONALE :

- **Python chosen** for simplicity and readability
- **Pygame used** for interactive visualization
- *A algorithm implemented** for efficient pathfinding
- Grid structure used to simulate real-world navigation

This is similar to the design choice of CSV simplicity in the reference project , but adapted for AI visualization.

IMPLEMENTATION DETAILS :

The project is implemented using Python with the Pygame library.

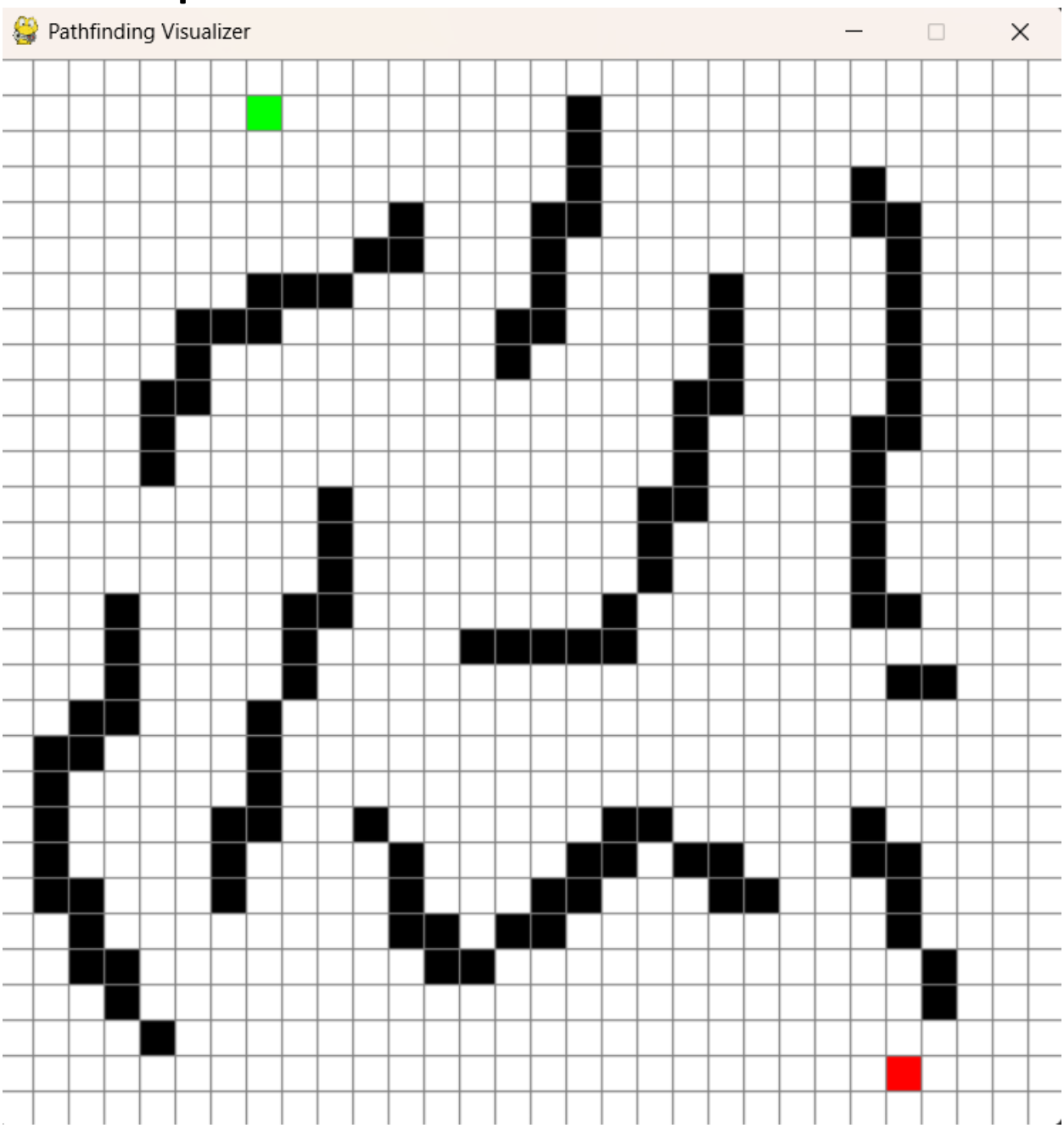
- Each grid cell is represented as a node (Spot class)
- Algorithms use:
  - Queue (BFS)
  - Stack (DFS)
  - Priority Queue (A*)
- Heuristic used: Manhattan Distance

SCREENSHOTS / RESULTS :

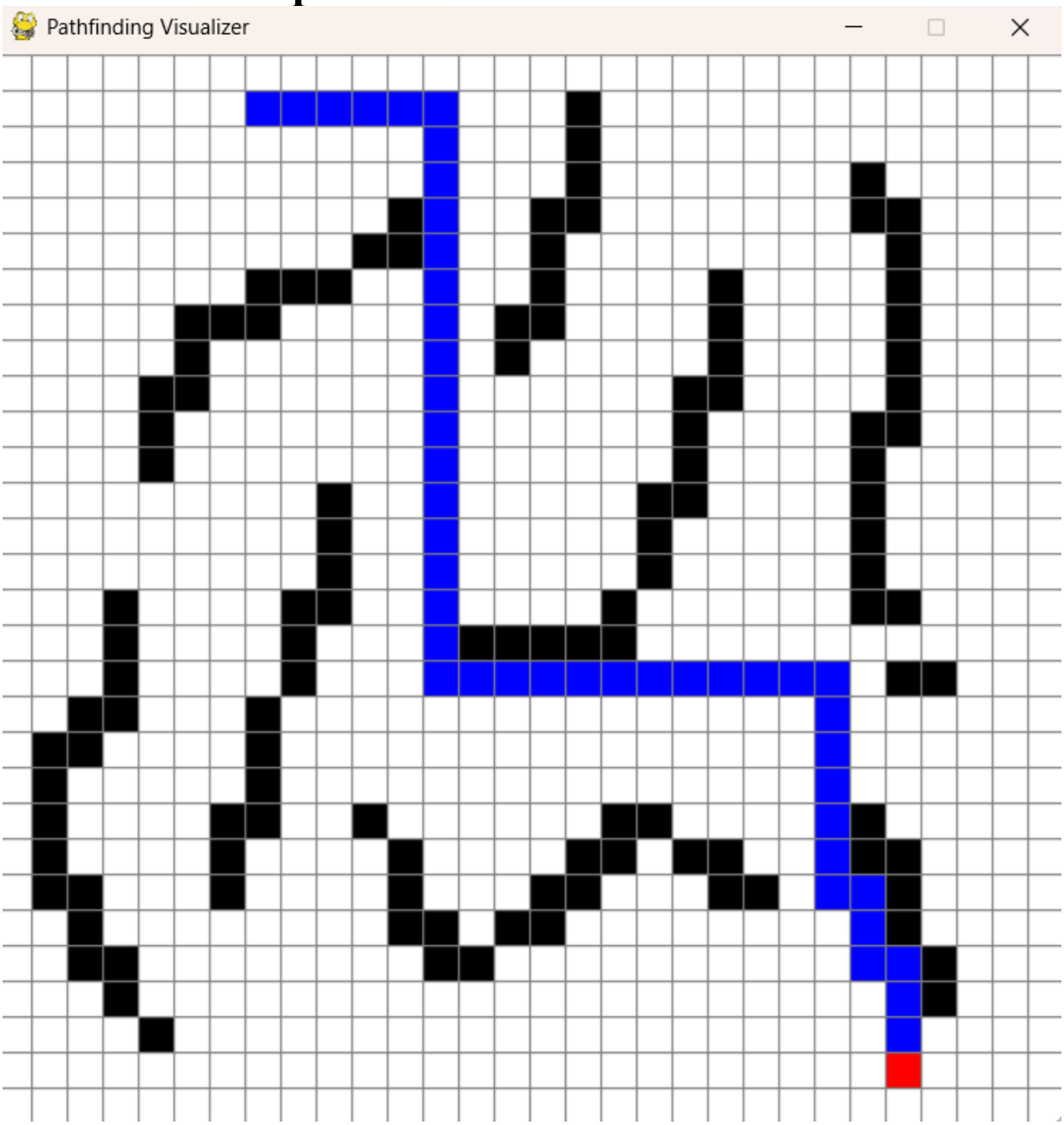
The system displays:

- Grid with start, end, and obstacles
- Algorithm exploration (visited nodes highlighted)
- Final shortest path

1. Grid Setup :



2. Final Path Output :



TESTING APPROACH

- Manual testing with multiple grid configurations
- Tested different obstacle placements
- Verified:
  - Correct path generation
  - Algorithm behavior differences
- Edge cases tested:
  - No path available
  - Start equals end

This approach is similar to manual testing in the reference project .

CHALLENGES FACED

- Implementing real-time visualization
- Managing grid updates efficiently
- Ensuring correct algorithm behavior
- Handling user input dynamically

LEARNINGS & KEY TAKEAWAYS

- Understanding search algorithms (BFS, DFS, A*)
- Importance of heuristics in AI
- Difference between informed and uninformed search
- Practical implementation of theoretical AI concepts

FUTURE ENHANCEMENTS

- Add Dijkstra's Algorithm
- Introduce weighted paths (traffic simulation)
- Add GUI buttons instead of keyboard controls
- Integrate real-world maps
- Display performance metrics

REFERENCES

- Python Documentation
- Pygame Documentation
- AI/ML Course Notes
- Online Tutorials